LEVERA · POWERED BY SUN PRODUCTION · TECHNICAL RECOMMENDATION

Website backend and optimization plan

What the bilingual LEVERA site needs behind it before launch, how to build it with the least moving parts, and what to improve on the current frontend.

Prepared: 28 October 2026

Applies to: React (Vite) build in `levera-website`

Status: recommendation only, no code changed

01 Summary

The site is a **static React frontend**. All text lives in one translation file, and every interactive part (language switch, industry picker, calculator, report demo, FAQ) runs in the browser. Those parts need no server.

Only one thing on the site needs a backend: **the hiring-request form and everything that follows it.** Today the form checks the fields and shows the thank-you screen, but nothing is sent, stored or emailed.

Recommendation

- **Do not build a custom server or database application.** Add one small serverless function (`/api/inquiry`) and connect ready-made services: a CRM, a transactional email service, a booking tool and spam protection.
- This covers everything in section 08 of the client brief ("Build now") and nothing in its "Do not build" list.
- Before launch, also fix the **missing legal pages, favicon and domain settings**, and take the quick performance wins in section 06. The biggest is that the industry picker downloads about 400 KB of full-size photos to show 32-pixel thumbnails.

02 What exists today

Area	Current state
Pages	`/en/` and `/ko/`, each with its own title, description, language tag and hreflang links. `/` sends visitors to their language.
Inquiry form	8 fields plus a privacy checkbox and an optional marketing checkbox. Validated in the browser. **Submits nowhere.** The thank-you screen says "a confirmation email is on its way" and "Preview only".
Form draft	Saved in the visitor's browser (`localStorage`), including name, email and phone, and **never cleared**, not even after submitting.
Spam protection	A hidden "honeypot" field only. No CAPTCHA and no rate limiting.
Analytics	None.

Legal and contact	Privacy policy and Terms link to the top of the page (placeholders). `hello@levera.co` is a mailto link but hasn't been verified.
Domain	`SITE_URL` in `vite.config.js` is a guess (`https://levera.co`). It's used for canonical and hreflang links.
Favicon	Missing: every page load requests `/favicon.ico` and gets a 404.

03 Backend the site needs

This follows the brief's flow: save lead → notify LEVERA → localized thank-you → same-language confirmation email → offer consultation booking.

Capability	What it does	Recommended way	Priority
Form endpoint	Receives the form, re-validates every field on the server, rejects spam, then hands off to the services below.	One serverless function, `POST /api/inquiry`, on the same host as the site.	Must
Lead storage / CRM	Stores each lead with locale (EN/KO), source and status. Tracks the recruiting pipeline from the brief.	A CRM with an API (e.g. HubSpot free CRM, Pipedrive, or Airtable for a simple start).	Must
Team alert	Tells LEVERA immediately when a request arrives.	Email to the team inbox, optionally Slack. Sent by the same function.	Must
Confirmation email	Sends the lead the confirmation from the brief in their language, with a booking link.	Transactional email service (e.g. Resend, Postmark, SendGrid) with one EN and one KO template. The sending domain needs SPF, DKIM and DMARC.	Must
Consultation booking	Lets the lead pick a time. Follow-ups stop once they've booked.	Calendly or Cal.com link on the thank-you screen and in the email. A booking webhook moves the lead to "Consultation".	High
Spam protection	Blocks bots without annoying real users.	Keep the honeypot. Add Cloudflare Turnstile (invisible) verified on the server, plus basic rate limiting per IP.	Must
Privacy controls	Proves consent and honours opt-outs.	Store consent time and privacy-policy version. Keep marketing opt-in separate with one-click unsubscribe. Write a process for deletion requests and a retention period.	Must
Attribution & analytics	Shows which channels bring leads and how far visitors scroll.	Capture UTM parameters, referrer and landing page in hidden fields. Use privacy-friendly analytics (e.g. Plausible), or GA4 with a consent banner.	High

Legal pages	Privacy policy and Terms pages, in both languages.	Static pages in the same app, using text supplied by the client.	Must

Out of scope (from the brief's "Do not build" list)

No client dashboard or login, payroll, recurring subscriptions, time tracker, AI agents, live task data feeds or employee management system. The weekly-report and calculator sections are illustrations, so they need **no data fetching**.

Recruiting statuses to set up in the CRM

New inquiry → Consultation → Needs confirmed → Searching → Screening → Shortlist → Interviews → Hired → QA weeks 1–4 → Closed

Only "New inquiry" and "Consultation" are set automatically, by the form and the booking webhook. The LEVERA team moves leads through the rest by hand in the CRM.

04 Recommended architecture

Visitor submits form → /api/inquiry (serverless) → Verify Turnstile + validate → Create / update lead in CRM → Email: alert to LEVERA → Email: EN/KO confirmation + booking link → Thank-you screen + "Book a time"

Lead books a time → Booking tool webhook → /api/booking-hook → CRM status = Consultation, follow-ups stop

Option A: managed services (recommended)

- Static host with functions: Vercel, Netlify or Cloudflare Pages
- CRM with API, e.g. HubSpot free CRM
- Resend or Postmark for EN/KO emails
- Calendly or Cal.com for booking
- Cloudflare Turnstile for spam

Least code, and the LEVERA team works in a familiar CRM. Free or low tiers usually cover early volume; check current pricing.

Option B: own database

- Same host and functions
- Supabase or another Postgres database as the lead store
- Small admin view or Airtable sync for the team
- Same email, booking and spam services

More control over the data, but more to build and maintain. This only makes sense if the client doesn't want a CRM.

Lead record (fields to store)

Field	Notes
name, business, email, phone	From the form. Phone is optional. Store email lower-cased and use it to match repeat requests.
position, budget	The form sends a list index. **Store the English label** so reports read the same whichever language the visitor used.
requirements	Free text, with a length limit (e.g. 2,000 characters).
locale	`en` or `ko`. Picks the email template and the language for later contact.
privacy_consent, consent_at, policy_version	Required. Records proof of consent.
marketing_opt_in, unsubscribed_at	Optional. Separate from the privacy consent.
utm_source / medium / campaign, referrer, landing_path	For attribution.
status, created_at, booked_at	Pipeline tracking.

API contract

```
POST /api/inquiry
{ name, business, email, phone?, role: 0-5, budget: 0-4, requirements?,
  privacy: true, marketing: bool, locale: "en"|"ko",
  utm: {source, medium, campaign}, referrer, landing, website: "" (honeypot),
  turnstileToken }

200 { ok: true, bookingUrl }                      → show thank-you + "Book a time"
400 { ok: false, errors: { email: "invalid" } }   → show field errors in current language
429 { ok: false, error: "rate_limited" }          → "Please try again in a minute"
500 { ok: false }                                 → "Something went wrong" + email fallback
```

05 Frontend changes when the backend is connected

#	Change	Priority
1	In `Book.jsx`, replace the local "mark as sent" step with a real request to `/api/inquiry`. Disable the button while sending.	Must
2	Add EN/KO messages for network failure and rate limiting. Keep the visitor's entries if sending fails.	Must
3	Remove the "Preview only" line, and only say "a confirmation email is on its way" once one really is.	Must
4	**Clear the saved draft after a successful send.** Better still, don't save email and phone in the browser at all. On a shared computer, the next person can currently see them.	Must
5	Add a "Book a time" button on the thank-you screen, using the returned booking link.	High
6	Add the Turnstile widget, and send the locale, UTM parameters, referrer and landing path with the form.	High
7	Link the privacy policy and terms to real `/en/privacy` and `/ko/privacy` pages. Email links should keep the visitor's language, e.g. `/ko/…`.	Must

06 Optimization opportunities

Measured on the current production build:

212 KB
JavaScript (≈70 KB compressed)

35 KB
CSS

776 KB
Images, 17 WebP files

4
Google font families, 2 of them Korean

Performance

Finding	Recommendation	Impact
The industry picker shows six 32 px thumbnails using the **full 1100 × 1183 hero photos** (24–122 KB each, about 400 KB in total), all downloaded on page load.	Make 64 px thumbnails (about 2–3 KB each). Use `srcset` at 600 and 1100 px widths for the hero photo.	High
The footer wordmark is a 2400 px image (64 KB) shown at 12% opacity. The background mark in the form section is 61 KB.	Export both as SVG (a few KB each), or at about half the size.	Medium
Images below the first screen load eagerly, except the people photos and the footer wordmark.	Add `loading="lazy"` and width/height to every below-the-fold image.	Medium
English pages also download the Korean fonts' stylesheet, and every font comes from Google at run time.	Load Noto Sans/Serif KR only on `/ko/` pages. Self-host Fraunces and Geist and preload them.	High

The `/en/` and `/ko/` HTML files have no visible content until JavaScript runs.	Pre-render each language page to full HTML at build time (e.g. a static site generation step with `react-dom/server`). Faster first paint, and better SEO.	High
The hero keeps rotating industries every 7 seconds, even when off-screen or in a background tab.	Pause the rotation when the hero isn't visible or the tab is hidden.	Low
React accounts for most of the 70 KB of compressed JavaScript.	Fine as it is. Swapping in Preact would save about 30 KB, but only consider it if speed scores demand it.	Low
Hosting cache settings aren't defined.	Cache hashed `/assets/*` for a year (immutable), images for a long time, and revalidate the HTML.	Medium

SEO and sharing

- Confirm the real domain and set `SITE_URL`. The canonical and hreflang links depend on it.
- Add `sitemap.xml` (listing both language pages) and `robots.txt`.
- Add Open Graph and Twitter tags with a 1200 × 630 share image per language.
- Add structured data: `Organization`, plus `FAQPage` built from the existing FAQ text.
- Add a favicon and an Apple touch icon (this also removes the current 404).

Accessibility

- The small grey text (`#7A827C` on white) has a contrast ratio of about **4:1**, below the 4.5:1 that WCAG AA requires for small text. It's used for fine print, labels and captions. Darken it slightly, e.g. to `#646D66`.
- Run an automated audit (axe or Lighthouse) on both languages, then a keyboard-only pass through the form and the tab controls.
- Translate the footer navigation's accessible label, which is currently fixed English text: "Footer".

Security (set at the host)

- HTTPS only, with HSTS. Add a Content Security Policy that allows only Google Fonts (or the self-hosted fonts), Turnstile and analytics.
- Set `X-Content-Type-Options`, `Referrer-Policy` and `Permissions-Policy` headers.
- Keep API keys only in the serverless function's environment variables, never in frontend code.

Code health

- Move the browser tests used during development into the project, so they run before every deploy.
- Add a linter (ESLint).
- Delete the `legacy/` folder (880 KB) once the client has approved the React version. It isn't deployed.
- Remove two images that are still deployed but no longer used after the redesign: `calm-morning.webp` and `logo-vertical.webp` (about 54 KB).
- Split `i18n.js` into one file per language once copy changes become frequent.

07 Needed from the client before launch

#	Item
1	Final domain, and access to its DNS (for the site and for email authentication).
2	Verified contact email address, and the inbox or Slack channel for new-lead alerts.
3	Choice of CRM and booking tool, or approval of the recommended ones.
4	Privacy policy and Terms text in English and Korean, with legal review. The brief's launch note also lists the service agreement, worker classification and recruiting regulations.
5	Analytics choice, and whether a cookie/consent banner is needed.
6	Wording approved for the EN/KO confirmation emails, and a sender name.
7	Native Korean review of all site and email copy.
8	Permission for the ~280-vehicle customer reference, if it's to be used.

08 Suggested order of work

Phase	Scope	Rough effort*
1. Launch-ready	Form endpoint, CRM connection, EN/KO emails, team alert, Turnstile and rate limiting, consent records, draft clearing, legal pages, favicon, domain and `SITE_URL`, security headers.	4–6 developer days
2. Fast and findable	Image thumbnails and `srcset`, SVG logo marks, lazy loading, fonts per language and self-hosted, pre-rendered HTML, sitemap, Open Graph tags, structured data, contrast fix.	2–4 developer days
3. Measure and automate	Analytics with consent, UTM reporting, booking webhook to stop follow-ups, optional follow-up email sequence, automated tests before each deploy.	2–3 developer days

*These are rough planning estimates, assuming the client's decisions in section 07 are made. They don't include legal review or copywriting time.

Final goal from the brief: the visitor understands, "You tell LEVERA what your business needs. We find, vet, and introduce the right person. You hire directly. We support the first four weeks." Every item above supports that message without adding software the business doesn't sell.